JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ObjectInputStream

java.lang.Object
    java.io.InputStream
        java.io.ObjectInputStream

All Implemented Interfaces:

Closeable, DataInput, ObjectInput, ObjectStreamConstants, AutoCloseable

---

```
public class ObjectInputStream
extends InputStream
implements ObjectInput, ObjectStreamConstants
```

An ObjectInputStream deserializes primitive data and objects previously written using an ObjectOutputStream.

Warning: Deserialization of untrusted data is inherently dangerous and should be avoided. Untrusted data should be carefully validated according to the "Serialization and Deserialization" section of the Secure Coding Guidelines for Java SE. Serialization Filtering describes best practices for defensive use of serial filters.

The key to disabling deserialization attacks is to prevent instances of arbitrary classes from being deserialized, thereby preventing the direct or indirect execution of their methods. `ObjectInputFilter` describes how to use filters and `ObjectInputFilter.Config` describes how to configure the filter and filter factory. Each stream has an optional deserialization filter to check the classes and resource limits during deserialization. The JVM-wide filter factory ensures that a filter can be set on every `ObjectInputStream` and every object read from the stream can be checked. The ObjectInputStream constructors invoke the filter factory to select the initial filter which may be updated or replaced by `setObjectInputFilter(ObjectInputFilter)`.

If an ObjectInputStream has a filter, the `ObjectInputFilter` can check that the classes, array lengths, number of references in the stream, depth, and number of bytes consumed from the input stream are allowed and if not, can terminate deserialization.

ObjectOutputStream and ObjectInputStream can provide an application with persistent storage for graphs of objects when used with a FileOutputStream and FileInputStream respectively. ObjectInputStream is used to recover those objects previously serialized. Other uses include passing objects between hosts using a socket stream or for marshaling and unmarshaling arguments and parameters in a remote communication system.

ObjectInputStream ensures that the types of all objects in the graph created from the stream match the classes present in the Java Virtual Machine. Classes are loaded as required using the standard mechanisms.

Only objects that support the java.io.Serializable or java.io.Externalizable interface can be read from streams.

The method `readObject` is used to read an object from the stream. Java's safe casting should be used to get the desired type. In Java, strings and arrays are objects and are treated as objects during serialization. When read they need to be cast to the expected type.

Primitive data types can be read from the stream using the appropriate method on DataInput.

The default deserialization mechanism for objects restores the contents of each field to the value and type it had when it was written. Fields declared as transient or static are ignored by the deserialization process. References to other objects cause those objects to be read from the stream as necessary. Graphs of objects are restored correctly using a reference sharing mechanism. New objects are always allocated when deserializing, which prevents existing objects from being overwritten.

Reading an object is analogous to running the constructors of a new object. Memory is allocated for the object and initialized to zero (NULL). No-arg constructors are invoked for the non-serializable classes and then the fields of the serializable classes are restored from the stream starting with the serializable class closest to java.lang.object and finishing with the object's most specific class.

For example to read from a stream as written by the example in `ObjectOutputStream`:

```
    try (FileInputStream fis = new FileInputStream("t.tmp");
         ObjectInputStream ois = new ObjectInputStream(fis)) {
        String label = (String) ois.readObject();
        LocalDateTime dateTime = (LocalDateTime) ois.readObject();
        // Use label and dateTime
    } catch (Exception ex) {
        // handle exception
    }
```

Classes control how they are serialized by implementing either the java.io.Serializable or java.io.Externalizable interfaces.

Implementing the Serializable interface allows object serialization to save and restore the entire state of the object and it allows classes to evolve between the time the stream is written and the time it is read. It automatically traverses references between objects, saving and restoring entire graphs.

Serializable classes that require special handling during the serialization and deserialization process should implement methods with the following signatures:

```
    private void writeObject(java.io.ObjectOutputStream stream)
        throws IOException;
    private void readObject(java.io.ObjectInputStream stream)
        throws IOException, ClassNotFoundException;
    private void readObjectNoData()
        throws ObjectStreamException;
```

The method name, modifiers, return type, and number and type of parameters must match exactly for the method to be used by serialization or deserialization. The methods should only be declared to throw checked exceptions consistent with these signatures.

The readObject method is responsible for reading and restoring the state of the object for its particular class using data written to the stream by the corresponding writeObject method. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is restored by reading data from the ObjectInputStream for the individual fields and making assignments to the appropriate fields of the object. Reading primitive data types is supported by DataInput.

Any attempt to read object data which exceeds the boundaries of the custom data written by the corresponding writeObject method will cause an OptionalDataException to be thrown with an eof field value of true. Non-object reads which exceed the end of the allotted data will reflect the end of data in the same way that they would indicate the end of the stream: bytewise reads will return -1 as the byte read or number of bytes read, and primitive reads will throw EOFExceptions. If there is no corresponding writeObject method, then the end of default serialized data marks the end of the allotted data.

Primitive and object read calls issued from within a readExternal method behave in the same manner--if the stream is already positioned at the end of data written by the corresponding writeExternal method, object reads will throw OptionalDataExceptions with eof set to true, bytewise reads will return -1, and primitive reads will throw EOFExceptions. Note that this behavior does not hold for streams written with the old `ObjectStreamConstants.PROTOCOL_VERSION_1` protocol, in which the end of

data written by writeExternal methods is not demarcated, and hence cannot be detected.

The readObjectNoData method is responsible for initializing the state of the object for its particular class in the event that the serialization stream does not list the given class as a superclass of the object being deserialized. This may occur in cases where the receiving party uses a different version of the deserialized instance's class than the sending party, and the receiver's version extends classes that are not extended by the sender's version. This may also occur if the serialization stream has been tampered; hence, readObjectNoData is useful for initializing deserialized objects properly despite a "hostile" or incomplete source stream.

Serialization does not read or assign values to the fields of any object that does not implement the java.io.Serializable interface. Subclasses of Objects that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state of the non-serializable class. It is frequently the case that the fields of that class are accessible (public, package, or protected) or that there are get and set methods that can be used to restore the state.

Any exception that occurs while deserializing an object will be caught by the ObjectInputStream and abort the reading process.

Implementing the Externalizable interface allows the object to assume complete control over the contents and format of the object's serialized form. The methods of the Externalizable interface, writeExternal and readExternal, are called to save and restore the objects state. When implemented by a class they can write and read their own state using all of the methods of ObjectOutput and ObjectInput. It is the responsibility of the objects to handle any versioning that occurs.

Enum constants are deserialized differently than ordinary serializable or externalizable objects. The serialized form of an enum constant consists solely of its name; field values of the constant are not transmitted. To deserialize an enum constant, ObjectInputStream reads the constant name from the stream; the deserialized constant is then obtained by calling the static method `Enum.valueOf(Class, String)` with the enum constant's base type and the received constant name as arguments. Like other serializable or externalizable objects, enum constants can function as the targets of back references appearing subsequently in the serialization stream. The process by which enum constants are deserialized cannot be customized: any class-specific readObject, readObjectNoData, and readResolve methods defined by enum types are ignored during deserialization. Similarly, any serialPersistentFields

or serialVersionUID field declarations are also ignored--all enum types have a fixed serialVersionUID of 0L.

Records are serialized differently than ordinary serializable or externalizable objects. During deserialization the record's canonical constructor is invoked to construct the record object. Certain serialization-related methods, such as readObject and writeObject, are ignored for serializable records. See *Java Object Serialization Specification,* Section 1.13, "Serialization of Records" for additional information.

Since:

1.1

External Specifications

Java Object Serialization Specification

See Also:

`DataInput`,
`ObjectOutputStream`,
`Serializable`,
Java Object Serialization Specification, Section 3, "Object Input Classes"

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	**ObjectInputStream.GetFie**	Provide access to the persistent fields read from the input stream.

Field Summary

Fields declared in interface ObjectStreamConstants

`baseWireHandle`, `PROTOCOL_VERSION_1`, `PROTOCOL_VERSION_2`, `SC_BLOCK_DATA`, `SC_ENUM`, `SC_EXTERNALIZABLE`, `SC_SERIALIZABLE`, `SC_WRITE_METHOD`, `SERIAL_FILTER_PERMISSION`, `STREAM_MAGIC`, `STREAM_VERSION`, `SUBCLASS_IMPLEMENTATION_PERMISSION`, `SUBSTITUTION_PERMISSION`, `TC_ARRAY`, `TC_BASE`, `TC_BLOCKDATA`, `TC_BLOCKDATALONG`, `TC_CLASS`, `TC_CLASSDESC`, `TC_ENDBLOCKDATA`, `TC_ENUM`, `TC_EXCEPTION`, `TC_LONGSTRING`, `TC_MAX`, `TC_NULL`, `TC_OBJECT`, `TC_PROXYCLASSDESC`, `TC_REFERENCE`, `TC_RESET`, `TC_STRING`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`ObjectInputStream()`	Provide a way for subclasses that are completely reimplementing ObjectInputStream to not have to allocate private data just used by this implementation of ObjectInputStream.
	`ObjectInputStream (InputStream in)`	Creates an ObjectInputStream that reads from the specified InputStream.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	`available()`	Returns the number of bytes that can be read without blocking.
void	`close()`	Closes this input stream and releases any system resources associated with the stream.
void	`defaultReadObject()`	Read the non-static and non-transient fields of the current class from this stream.
protected boolean	`enableResolveObject (boolean enable)`	Enables the stream to do replacement of objects read from the stream.
	`getObjectInputFilter()`	

final ObjectInputFilter		Returns the deserialization filter for this stream.
int	**read**()	Reads a byte of data.
int	**read**(byte[] buf, int off, int len)	Reads into an array of bytes.
boolean	**readBoolean**()	Reads in a boolean.
byte	**readByte**()	Reads an 8-bit byte.
char	**readChar**()	Reads a 16-bit char.
protected ObjectStreamClass	**readClassDescriptor**()	Read a class descriptor from the serialization stream.
double	**readDouble**()	Reads a 64-bit double.
ObjectInputStream.Ge	**readFields**()	Reads the persistent fields from the stream and makes them available by name.
float	**readFloat**()	Reads a 32-bit float.
void	**readFully**(byte[] buf)	Reads bytes, blocking until all bytes are read.
void	**readFully**(byte[] buf, int off, int len)	Reads bytes, blocking until all bytes are read.
int	**readInt**()	Reads a 32-bit int.
String	**readLine**()	**Deprecated.** This method does not properly convert bytes to characters.
long	**readLong**()	Reads a 64-bit long.
final Object	**readObject**()	Read an object from the ObjectInputStream.
protected Object	**readObjectOverride**()	This method is called by trusted subclasses of

		ObjectInputStream that constructed ObjectInputStream using the protected no-arg constructor.
short	**readShort**()	Reads a 16-bit short.
protected void	**readStreamHeader**()	The readStreamHeader method is provided to allow subclasses to read and verify their own stream headers.
Object	**readUnshared**()	Reads an "unshared" object from the ObjectInputStream.
int	**readUnsignedByte**()	Reads an unsigned 8-bit byte.
int	**readUnsignedShort**()	Reads an unsigned 16-bit short.
String	**readUTF**()	Reads a String in modified UTF-8 format.
void	**registerValidation**(ObjectInputValidation o int prio)	Register an object to be validated before the graph is returned.
protected Class<?>	**resolveClass**(ObjectStreamClass desc)	Load the local class equivalent of the specified stream class description.
protected Object	**resolveObject**(Object obj)	This method will allow trusted subclasses of ObjectInputStream to substitute one object for another during deserialization.
protected Class<?>	**resolveProxyClass**(String[] interfaces)	Returns a proxy class that implements the

		interfaces named in a proxy class descriptor; subclasses may implement this method to read custom data from the stream along with the descriptors for dynamic proxy classes, allowing them to use an alternate loading mechanism for the interfaces and the proxy class.
final void	**setObjectInputFilter**(ObjectInputFilter filte	Set the deserialization filter for the stream.
int	**skipBytes**(int len)	Skips bytes.

Methods declared in class InputStream

mark, markSupported, nullInputStream, read, readAllBytes, readNBytes, readNBytes, reset, skip, skipNBytes, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface ObjectInput

read, skip

Constructor Details

ObjectInputStream

```
public ObjectInputStream(InputStream in)
                  throws IOException
```

Creates an ObjectInputStream that reads from the specified InputStream. A serialization stream header is read from the stream and verified. This

constructor will block until the corresponding ObjectOutputStream has written and flushed the header.

The constructor initializes the deserialization filter to the filter returned by invoking the serial filter factory returned from `ObjectInputFilter.Config.getSerialFilterFactory()` with `null` for the current filter and the static JVM-wide filter for the requested filter. If the serial filter or serial filter factory properties are invalid an `IllegalStateException` is thrown. When the filter factory `apply` method is invoked it may throw a runtime exception preventing the `ObjectInputStream` from being constructed.

Parameters:

`in` - input stream to read from

Throws:

`StreamCorruptedException` - if the stream header is incorrect

`IOException` - if an I/O error occurs while reading stream header

`IllegalStateException` - if the initialization of `ObjectInputFilter.Config` fails due to invalid serial filter or serial filter factory properties.

`NullPointerException` - if `in` is `null`

See Also:

`ObjectInputStream()`,
`readFields()`,
`ObjectOutputStream(OutputStream)`

ObjectInputStream

```
protected ObjectInputStream()
                    throws IOException
```

Provide a way for subclasses that are completely reimplementing ObjectInputStream to not have to allocate private data just used by this implementation of ObjectInputStream.

The constructor initializes the deserialization filter to the filter returned by invoking the serial filter factory returned from `ObjectInputFilter.Config.getSerialFilterFactory()` with `null` for the current filter and the static JVM-wide filter for the requested filter. If the serial filter or serial filter factory properties are invalid an `IllegalStateException` is thrown. When the filter factory `apply` method is invoked it may throw a runtime exception preventing the `ObjectInputStream` from being constructed.

Throws:

IOException - if an I/O error occurs while creating this stream

IllegalStateException - if the initialization of ObjectInputFilter.Config fails due to invalid serial filter or serial filter factory properties.

Method Details

readObject

```
public final Object readObject()
                        throws IOException,
                               ClassNotFoundException
```

Read an object from the ObjectInputStream. The class of the object, the signature of the class, and the values of the non-transient and non-static fields of the class and all of its supertypes are read. Default deserializing for a class can be overridden using the writeObject and readObject methods. Objects referenced by this object are read transitively so that a complete equivalent graph of objects is reconstructed by readObject.

The root object is completely restored when all of its fields and the objects it references are completely restored. At this point the object validation callbacks are executed in order based on their registered priorities. The callbacks are registered by objects (in the readObject special methods) as they are individually restored.

The deserialization filter, when not `null`, is invoked for each object (regular or class) read to reconstruct the root object. See setObjectInputFilter for details.

Exceptions are thrown for problems with the InputStream and for classes that should not be deserialized. All exceptions are fatal to the InputStream and leave it in an indeterminate state; it is up to the caller to ignore or recover the stream state.

Specified by:

readObject in interface ObjectInput

Returns:

the object read from the stream

Throws:

ClassNotFoundException - Class of a serialized object cannot be found.

InvalidClassException - Something is wrong with a class used by deserialization.

StreamCorruptedException - Control information in the stream is inconsistent.

`OptionalDataException` - Primitive data was found in the stream instead of objects.

`IOException` - Any of the usual Input/Output related exceptions.

readObjectOverride

```
protected Object readObjectOverride()
                            throws IOException,
                                   ClassNotFoundException
```

This method is called by trusted subclasses of ObjectInputStream that constructed ObjectInputStream using the protected no-arg constructor. The subclass is expected to provide an override method with the modifier "final".

Returns:

the Object read from the stream.

Throws:

`ClassNotFoundException` - Class definition of a serialized object cannot be found.

`OptionalDataException` - Primitive data was found in the stream instead of objects.

`IOException` - if I/O errors occurred while reading from the underlying stream

Since:

1.2

See Also:

`ObjectInputStream()`, `readObject()`

readUnshared

```
public Object readUnshared()
                    throws IOException,
                           ClassNotFoundException
```

Reads an "unshared" object from the ObjectInputStream. This method is identical to readObject, except that it prevents subsequent calls to readObject and readUnshared from returning additional references to the deserialized instance obtained via this call. Specifically:

- If readUnshared is called to deserialize a back-reference (the stream representation of an object which has been written previously to the stream), an ObjectStreamException will be thrown.

- If readUnshared returns successfully, then any subsequent attempts to deserialize back-references to the stream handle deserialized by readUnshared will cause an ObjectStreamException to be thrown.

Deserializing an object via readUnshared invalidates the stream handle associated with the returned object. Note that this in itself does not always guarantee that the reference returned by readUnshared is unique; the deserialized object may define a readResolve method which returns an object visible to other parties, or readUnshared may return a Class object or enum constant obtainable elsewhere in the stream or through external means. If the deserialized object defines a readResolve method and the invocation of that method returns an array, then readUnshared returns a shallow clone of that array; this guarantees that the returned array object is unique and cannot be obtained a second time from an invocation of readObject or readUnshared on the ObjectInputStream, even if the underlying data stream has been manipulated.

The deserialization filter, when not `null`, is invoked for each object (regular or class) read to reconstruct the root object. See `setObjectInputFilter` for details.

Returns:

reference to deserialized object

Throws:

`ClassNotFoundException` - if class of an object to deserialize cannot be found

`StreamCorruptedException` - if control information in the stream is inconsistent

`ObjectStreamException` - if object to deserialize has already appeared in stream

`OptionalDataException` - if primitive data is next in stream

`IOException` - if an I/O error occurs during deserialization

Since:

1.4

defaultReadObject

```
public void defaultReadObject()
                     throws IOException,
                            ClassNotFoundException
```

Read the non-static and non-transient fields of the current class from this stream. This may only be called from the readObject method of the class

being deserialized. It will throw the NotActiveException if it is called otherwise.

Throws:

`ClassNotFoundException` - if the class of a serialized object could not be found.

`IOException` - if an I/O error occurs.

`NotActiveException` - if the stream is not currently reading objects.

readFields

```
public ObjectInputStream.GetField readFields()
                                        throws IOException,
                                               ClassNotFoundException
```

Reads the persistent fields from the stream and makes them available by name.

Returns:

the `GetField` object representing the persistent fields of the object being deserialized

Throws:

`ClassNotFoundException` - if the class of a serialized object could not be found.

`IOException` - if an I/O error occurs.

`NotActiveException` - if the stream is not currently reading objects.

Since:

1.2

registerValidation

```
public void registerValidation(ObjectInputValidation obj,
                               int prio)
                        throws NotActiveException,
                               InvalidObjectException
```

Register an object to be validated before the graph is returned. While similar to resolveObject these validations are called after the entire graph has been reconstituted. Typically, a readObject method will register the object with the stream so that when all of the objects are restored a final set of validations can be performed.

Parameters:

`obj` - the object to receive the validation callback.

`prio` - controls the order of callbacks; zero is a good default. Use higher numbers to be called back earlier, lower numbers for later callbacks. Within a priority, callbacks are processed in no particular order.

Throws:

`NotActiveException` - The stream is not currently reading objects so it is invalid to register a callback.

`InvalidObjectException` - The validation object is null.

resolveClass

```
protected Class<?> resolveClass(ObjectStreamClass desc)
                         throws IOException,
                                ClassNotFoundException
```

Load the local class equivalent of the specified stream class description. Subclasses may implement this method to allow classes to be fetched from an alternate source.

The corresponding method in `ObjectOutputStream` is `annotateClass`. This method will be invoked only once for each unique class in the stream. This method can be implemented by subclasses to use an alternate loading mechanism but must return a `Class` object. Once returned, if the class is not an array class, its serialVersionUID is compared to the serialVersionUID of the serialized class, and if there is a mismatch, the deserialization fails and an `InvalidClassException` is thrown.

The default implementation of this method in `ObjectInputStream` returns the result of calling

```
    Class.forName(desc.getName(), false, loader)
```

where `loader` is the first class loader on the current thread's stack (starting from the currently executing method) that is neither the platform class loader nor its ancestor; otherwise, `loader` is the *platform class loader*. If this call results in a `ClassNotFoundException` and the name of the passed `ObjectStreamClass` instance is the Java language keyword for a primitive type or void, then the `Class` object representing that primitive type or void will be returned (e.g., an `ObjectStreamClass` with the name `"int"` will be resolved to `Integer.TYPE`). Otherwise, the `ClassNotFoundException` will be thrown to the caller of this method.

Parameters:

`desc` - an instance of class `ObjectStreamClass`

Returns:

a `Class` object corresponding to `desc`

Throws:

`IOException` - any of the usual Input/Output exceptions.

`ClassNotFoundException` - if class of a serialized object cannot be found.

resolveProxyClass

```
protected Class<?> resolveProxyClass(String[] interfaces)
                                throws IOException,
                                       ClassNotFoundException
```

Returns a proxy class that implements the interfaces named in a proxy class descriptor; subclasses may implement this method to read custom data from the stream along with the descriptors for dynamic proxy classes, allowing them to use an alternate loading mechanism for the interfaces and the proxy class.

This method is called exactly once for each unique proxy class descriptor in the stream.

The corresponding method in `ObjectOutputStream` is `annotateProxyClass`. For a given subclass of `ObjectInputStream` that overrides this method, the `annotateProxyClass` method in the corresponding subclass of `ObjectOutputStream` must write any data or objects read by this method.

The default implementation of this method in `ObjectInputStream` returns the result of calling `Proxy.getProxyClass` with the list of `Class` objects for the interfaces that are named in the `interfaces` parameter. The `Class` object for each interface name `i` is the value returned by calling

```
    Class.forName(i, false, loader)
```

where `loader` is the first class loader on the current thread's stack (starting from the currently executing method) that is neither the platform class loader nor its ancestor; otherwise, `loader` is the *platform class loader*. Unless any of the resolved interfaces are non-public, this same value of `loader` is also the class loader passed to `Proxy.getProxyClass`; if non-public interfaces are present, their class loader is passed instead (if more than one non-public interface class loader is encountered, an `IllegalAccessError` is thrown). If `Proxy.getProxyClass` throws an `IllegalArgumentException`, `resolveProxyClass` will throw a `ClassNotFoundException` containing the `IllegalArgumentException`.

Parameters:

`interfaces` - the list of interface names that were deserialized in the proxy class descriptor

Returns:

a proxy class for the specified interfaces

Throws:

`IOException` - any exception thrown by the underlying `InputStream`

`ClassNotFoundException` - if the proxy class or any of the named interfaces could not be found

Since:

1.3

See Also:

`ObjectOutputStream.annotateProxyClass(Class)`

resolveObject

```
protected Object resolveObject(Object obj)
                        throws IOException
```

This method will allow trusted subclasses of ObjectInputStream to substitute one object for another during deserialization. Replacing objects is disabled until enableResolveObject is called. The enableResolveObject method checks that the stream requesting to resolve object can be trusted. Every reference to serializable objects is passed to resolveObject. To ensure that the private state of objects is not unintentionally exposed only trusted streams may use resolveObject.

This method is called after an object has been read but before it is returned from readObject. The default resolveObject method just returns the same object.

When a subclass is replacing objects it must ensure that the substituted object is compatible with every field where the reference will be stored. Objects whose type is not a subclass of the type of the field or array element abort the deserialization by raising an exception and the object is not be stored.

This method is called only once when each object is first encountered. All subsequent references to the object will be redirected to the new object.

Parameters:

`obj` - object to be substituted

Returns:

the substituted object

Throws:

`IOException` - Any of the usual Input/Output exceptions.

enableResolveObject

```
protected boolean enableResolveObject(boolean enable)
```

Enables the stream to do replacement of objects read from the stream. When enabled, the `resolveObject(Object)` method is called for every object being deserialized.

Parameters:

`enable` - true for enabling use of `resolveObject` for every object being deserialized

Returns:

the previous setting before this method was invoked

readStreamHeader

```
protected void readStreamHeader()
                          throws IOException,
                                 StreamCorruptedException
```

The readStreamHeader method is provided to allow subclasses to read and verify their own stream headers. It reads and verifies the magic number and version number.

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`StreamCorruptedException` - if control information in the stream is inconsistent

readClassDescriptor

```
protected ObjectStreamClass readClassDescriptor()
                                            throws
IOException,
ClassNotFoundException
```

Read a class descriptor from the serialization stream. This method is called when the ObjectInputStream expects a class descriptor as the next item in the serialization stream. Subclasses of ObjectInputStream may override this method to read in class descriptors that have been written in non-standard formats (by subclasses of ObjectOutputStream which have overridden the `writeClassDescriptor` method). By default, this method reads class

descriptors according to the format defined in the Object Serialization specification.

Returns:

the class descriptor read

Throws:

`IOException` - If an I/O error has occurred.

`ClassNotFoundException` - If the Class of a serialized object used in the class descriptor representation cannot be found

Since:

1.3

See Also:

`ObjectOutputStream.writeClassDescriptor(java.io.ObjectStreamClass)`

read

```
public int read()
         throws IOException
```

Reads a byte of data. This method will block if no input is available.

Specified by:

`read` in interface `ObjectInput`

Specified by:

`read` in class `InputStream`

Returns:

the byte read, or -1 if the end of the stream is reached.

Throws:

`IOException` - if an I/O error occurs.

read

```
public int read(byte[] buf,
                int off,
                int len)
         throws IOException
```

Reads into an array of bytes. This method will block until some input is available. Consider using java.io.DataInputStream.readFully to read exactly 'length' bytes.

Specified by:

`read` in interface `ObjectInput`

Overrides:

`read` in class `InputStream`

Parameters:

`buf` - the buffer into which the data is read

`off` - the start offset in the destination array `buf`

`len` - the maximum number of bytes read

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`NullPointerException` - if `buf` is `null`.

`IndexOutOfBoundsException` - if `off` is negative, `len` is negative, or `len` is greater than `buf.length - off`.

`IOException` - If an I/O error has occurred.

See Also:

`DataInputStream.readFully(byte[],int,int)`

available

```
public int available()
             throws IOException
```

Returns the number of bytes that can be read without blocking.

Specified by:

`available` in interface `ObjectInput`

Overrides:

`available` in class `InputStream`

Returns:

the number of available bytes.

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Specified by:

close in interface ObjectInput

Overrides:

close in class InputStream

Throws:

IOException - if an I/O error occurs.

readBoolean

```
public boolean readBoolean()
                    throws IOException
```

Reads in a boolean.

Specified by:

readBoolean in interface DataInput

Returns:

the boolean read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readByte

```
public byte readByte()
              throws IOException
```

Reads an 8-bit byte.

Specified by:

readByte in interface DataInput

Returns:

the 8-bit byte read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readUnsignedByte

```
public int readUnsignedByte()
                    throws IOException
```

Reads an unsigned 8-bit byte.

Specified by:

readUnsignedByte in interface DataInput

Returns:

the 8-bit byte read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readChar

```
public char readChar()
             throws IOException
```

Reads a 16-bit char.

Specified by:

readChar in interface DataInput

Returns:

the 16-bit char read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readShort

```
public short readShort()
                throws IOException
```

Reads a 16-bit short.

Specified by:

readShort in interface DataInput

Returns:

the 16-bit short read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readUnsignedShort

```
public int readUnsignedShort()
                     throws IOException
```

Reads an unsigned 16-bit short.

Specified by:

readUnsignedShort in interface DataInput

Returns:

the 16-bit short read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readInt

```
public int readInt()
            throws IOException
```

Reads a 32-bit int.

Specified by:

readInt in interface DataInput

Returns:

the 32-bit integer read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readLong

```
public long readLong()
              throws IOException
```

Reads a 64-bit long.

Specified by:

readLong in interface DataInput

Returns:

the read 64-bit long.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readFloat

```
public float readFloat()
                throws IOException
```

Reads a 32-bit float.

Specified by:

readFloat in interface DataInput

Returns:

the 32-bit float read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readDouble

```
public double readDouble()
                  throws IOException
```

Reads a 64-bit double.

Specified by:

readDouble in interface DataInput

Returns:

the 64-bit double read.

Throws:

EOFException - If end of file is reached.

IOException - If other I/O error has occurred.

readFully

```
public void readFully(byte[] buf)
               throws IOException
```

Reads bytes, blocking until all bytes are read.

Specified by:

readFully in interface DataInput

Parameters:

buf - the buffer into which the data is read

Throws:

`NullPointerException` - If `buf` is `null`.

`EOFException` - If end of file is reached.

`IOException` - If other I/O error has occurred.

readFully

```
public void readFully(byte[] buf,
                      int off,
                      int len)
               throws IOException
```

Reads bytes, blocking until all bytes are read.

Specified by:

`readFully` in interface `DataInput`

Parameters:

`buf` - the buffer into which the data is read

`off` - the start offset into the data array `buf`

`len` - the maximum number of bytes to read

Throws:

`NullPointerException` - If `buf` is `null`.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `buf.length - off`.

`EOFException` - If end of file is reached.

`IOException` - If other I/O error has occurred.

skipBytes

```
public int skipBytes(int len)
              throws IOException
```

Skips bytes.

Specified by:

`skipBytes` in interface `DataInput`

Parameters:

`len` - the number of bytes to be skipped

Returns:

the actual number of bytes skipped.

Throws:

IOException - If an I/O error has occurred.

readLine

```
@Deprecated
public String readLine()
                throws IOException
```

Deprecated.
This method does not properly convert bytes to characters. see DataInputStream for the details and alternatives.

Reads in a line that has been terminated by a \n, \r, \r\n or EOF.

Specified by:

readLine in interface DataInput

Returns:

a String copy of the line.

Throws:

IOException - if there are I/O errors while reading from the underlying InputStream

readUTF

```
public String readUTF()
              throws IOException
```

Reads a String in modified UTF-8 format.

Specified by:

readUTF in interface DataInput

Returns:

the String.

Throws:

IOException - if there are I/O errors while reading from the underlying InputStream

UTFDataFormatException - if read bytes do not represent a valid modified UTF-8 encoding of a string

getObjectInputFilter

```
public final ObjectInputFilter getObjectInputFilter()
```

Returns the deserialization filter for this stream. The filter is the result of invoking the `JVM-wide filter factory` either by the constructor or the most recent invocation of `setObjectInputFilter`.

Returns:

the deserialization filter for the stream; may be null

Since:

9

setObjectInputFilter

```
public final void setObjectInputFilter(ObjectInputFilter filter)
```

Set the deserialization filter for the stream. The deserialization filter is set to the filter returned by invoking the JVM-wide filter factory with the current filter and the `filter` parameter. The current filter was set in the ObjectInputStream constructors by invoking the JVM-wide filter factory and may be `null`. setObjectInputFilter(ObjectInputFilter) This method} can be called once and only once before reading any objects from the stream; for example, by calling `readObject()` or `readUnshared()`.

It is not permitted to replace a `non-null` filter with a `null` filter. If the current filter is `non-null`, the value returned from the filter factory must be `non-null`.

The filter's `checkInput` method is called for each class and reference in the stream. The filter can check any or all of the class, the array length, the number of references, the depth of the graph, and the size of the input stream. The depth is the number of nested readObject calls starting with the reading of the root of the graph being deserialized and the current object being deserialized. The number of references is the cumulative number of objects and references to objects already read from the stream including the current object being read. The filter is invoked only when reading objects from the stream and not for primitives.

If the filter returns `Status.REJECTED`, `null` or throws a `RuntimeException`, the active `readObject` or `readUnshared` throws `InvalidClassException`, otherwise deserialization continues uninterrupted.

Implementation Requirements:

The filter, when not `null`, is invoked during `readObject` and `readUnshared` for each object (regular or class) in the stream. Strings are treated as primitives and do not invoke the filter. The filter is called for:

- each
  object reference previously deserialized from the stream (class is `null`, arrayLength is -1),
- each regular class (class is not `null`, arrayLength is -1),

- each interface class explicitly referenced in the stream (it is not called for interfaces implemented by classes in the stream),
- each interface of a dynamic proxy and the dynamic proxy class itself (class is not `null`, arrayLength is -1),
- each array is filtered using the array type and length of the array (class is the array type, arrayLength is the requested length),
- each object replaced by its class' `readResolve` method is filtered using the replacement object's class, if not `null`, and if it is an array, the arrayLength, otherwise -1,
- and each object replaced by `resolveObject` is filtered using the replacement object's class, if not `null`, and if it is an array, the arrayLength, otherwise -1.

When the `checkInput` method is invoked it is given access to the current class, the array length, the current number of references already read from the stream, the depth of nested calls to `readObject` or `readUnshared`, and the implementation dependent number of bytes consumed from the input stream.

Each call to `readObject` or `readUnshared` increases the depth by 1 before reading an object and decreases by 1 before returning normally or exceptionally. The depth starts at 1 and increases for each nested object and decrements when each nested call returns. The count of references in the stream starts at 1 and is increased before reading an object.

Parameters:

`filter` - the filter, may be null

Throws:

`IllegalStateException` - if an object has been read, if the filter factory returns `null` when the current filter is non-null, or if the filter has already been set.

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.